Contents

List of Tables i

List of Figures ii

List of Abbreviations iii

1 Introduction 1

1.1 Overview 1

1.2 Problem Statement 2

1.3 Objectives 4

1.4 Scope of the Project 6

1.5 Limitations 8

2 Literature Review 11

2.1 Introduction 11

2.2 Related Work 11

2.2.1 Optical Character Recognition for Medical Documents 11

2.2.2 Text Detection in Unstructured Images 12

2.2.3 Vision-Language Models for Document Understanding 13

2.2.4 Large Language Models for Clinical Text Processing 13

2.2.5 Mobile Health Reminder Systems and Asynchronous Processing 13

2.3 Research Gap 14

3 Requirement Analysis and System Specification 16

3.1 Introduction 16

3.2 Feasibility Study 16

3.2.1 Technical Feasibility 16

3.2.2 Operational Feasibility 17

3.2.3 Economic Feasibility 18
3.3 Software and Hardware Requirements 18
3.3.1 Hardware Requirements 18
3.3.2 Software Requirements 18
3.4 Functional Requirements 19
3.5 Non-Functional Requirements 22
3.5.1 NFR-01: Performance 22
3.5.2 NFR-02: Reliability 23
3.5.3 NFR-03: Security 23
3.5.4 NFR-04: Scalability 24
3.5.5 NFR-05: Usability 24
3.5.6 NFR-06: Maintainability 25
3.6 Validation 25

4 System Design 28
4.1 Introduction 28
4.2 System Architecture 29
4.2.1 Architectural Overview 29
4.2.2 System Components 29
4.3 UML Diagrams 32
4.3.1 Use Case Diagram 33
4.3.2 Class Diagram 33
4.3.3 Sequence Diagram 34
4.3.4 Activity Diagram 34
4.4 Database Design 34
4.4.1 Entity-Relationship Diagram 34
4.4.2 Table Schemas 35

5 Implementation, Discussions and Results 38
5.1 Introduction 38
5.2 Method of Implementation 38
5.2.1 Sprint 0 — Environment Setup and Requirements Finalisation . 38
5.2.2 Sprint 1 — User Management and Authentication 39

5.2.3 Sprint 2 — Prescription Ingestion and Status Tracking 40
5.2.4 Sprint 3 — AI Pipeline Integration 42
5.2.5 Sprint 4 — Mobile Reminder System and Dashboard 43
5.2.6 Sprint 5 — Testing, Documentation, and Deployment 44
5.3 AI Pipeline Implementation 44
5.3.1 Pipeline Overview 44
5.3.2 Stage 1 — Image Preprocessing 45
5.3.3 Stage 2 — CRAFT Text Detection 45
5.3.4 Stage 3 — PaddleOCR Text Extraction 45
5.3.5 Stage 4 — Qwen2-VL OCR Correction 45
5.3.6 Stage 5 — Qwen2.5-7B Structured Extraction 45
5.4 Discussions 46
5.4.1 Pipeline Accuracy and Design Trade-offs 46
5.4.2 Security Architecture Observations 47
5.4.3 Usability Findings 47
5.5 Results 47
5.5.1 Output Screenshots 47

6 Testing **49**
6.1 Introduction 49
6.2 Unit Testing 49
6.3 Integration Testing 49
6.4 System Testing 49
6.5 User Acceptance Testing 50
6.6 Performance and Security Testing 51
6.6.1 Performance Testing 51
6.6.2 Security Testing 51

7 Conclusion and Future Scope **55**
7.1 Conclusion 55
7.2 Future Enhancements 56

References **58**

List of Tables

3.1 Hardware Requirements 19
3.2 Software Requirements 19

4.1 Backend Modules and Responsibilities 31
4.2 AI Pipeline Stages 32
4.3 Database Table Schemas 36

6.1 Unit Test Cases — Backend 52
6.2 Integration Test Cases — Celery Task and AI Pipeline 53
6.3 User Acceptance Testing Results (N = 8) 54
6.4 Performance Test Results Against NFR-01 Targets 54
6.5 Security Test Cases 54

List of Figures

4.1 MedMate System Architecture 30
4.2 MedMate Use Case Diagram 33
4.3 Sequence Diagram — Prescription Upload and AI Processing 35
4.4 Activity Diagram — MedMate End-to-End User Workflow 37

5.1 MedMate Five-Stage AI Pipeline Data Flow 44

List of Abbreviations

Abbreviation	Expansion
AI	Artificial Intelligence
ML	Machine Learning
VLM	Vision-Language Model
OCR	Optical Character Recognition
LLM	Large Language Model
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
OTP	One-Time Password
DRF	Django REST Framework
GPU	Graphics Processing Unit
CUDA	Compute Unified Device Architecture
JSON	JavaScript Object Notation
UI	User Interface
UX	User Experience
SRS	Software Requirements Specification
DFD	Data Flow Diagram
ER	Entity Relationship
MFA	Multi-Factor Authentication
CLAHE	Contrast Limited Adaptive Histogram Equalization
INT4	4-bit Integer Quantisation
FP16	16-bit Floating Point Precision
CRNN	Convolutional Recurrent Neural Network
VQA	Visual Question Answering
ORM	Object Relational Mapper
PBKDF2	Password-Based Key Derivation Function 2

SMTP	Simple Mail Transfer Protocol
MIME	Multipurpose Internet Mail Extensions
WSGI	Web Server Gateway Interface
TTL	Time To Live
UAT	User Acceptance Testing

CHAPTER 1: Introduction

1.1 Overview

In most Indian outpatient clinics, the physician still hands the patient a handwritten slip of paper. Digital prescribing exists in large corporate hospitals, but in the government and semi-urban settings where the majority of consultations take place, the handwritten prescription is the only record of what was prescribed — and the patient is left to decipher it alone.

MedMate is a mobile-first healthcare application that solves this problem end-to-end: the patient photographs the prescription with a smartphone, uploads it through the MedMate app, and receives a structured list of medicines — with dosages, frequencies, and intake instructions — within approximately forty-five seconds. They can then configure dose reminder notifications for each medicine directly from the extracted data, without entering anything manually.

The system's core is a **five-stage AI pipeline**. An OpenCV preprocessing module normalises image quality for real-world capture conditions. A CRAFT text detector localises individual word regions across the prescription image. Classic PaddleOCR transcribes the text from each cropped region. A Qwen2-VL vision-language model corrects character-level OCR errors by cross-referencing the original image against the recognised text. Finally, a Qwen2.5-7B large language model resolves brand names, expands Latin abbreviations, and produces a validated JSON record for each identified medicine.

The backend is a Django REST API that manages user authentication via JSON Web Tokens with email one-time password verification. Prescription images and extracted records are persisted in a PostgreSQL relational database. AI processing tasks are dispatched to a Celery background worker through a Redis message broker — the mobile

client receives an immediate HTTP acknowledgement on upload and polls for results, avoiding HTTP timeouts during the forty-plus second pipeline execution. The Flutter mobile application handles local notification scheduling independently of network connectivity once reminders have been configured.

MedMate is built specifically for the Indian outpatient context, where handwritten prescriptions on pre-printed slips remain the norm and where patients frequently manage multiple medications without pharmacist counselling. The system currently supports English-language prescriptions and is optimised for commonly prescribed Indian brand names, regional abbreviations, and dosage conventions.

This document is the Software Requirements Specification for MedMate. It is intended for the project supervisor, department evaluators, and any future developers who may extend or maintain the system.

1.2 Problem Statement

Handwritten medical prescriptions remain the predominant medium for communicating treatment instructions from physicians to patients across a substantial proportion of clinical settings, particularly in developing economies where electronic prescribing systems have not been adopted. Despite advances in digital health infrastructure, the handwritten prescription persists due to its simplicity, low cost, and established clinical practice. This continued reliance introduces a series of interlinked problems that compromise patient safety, treatment adherence, and healthcare efficiency.

The first and most visible problem is illegibility. Physician handwriting exhibits high variability both within and across practitioners. Characters may be malformed, connected, or superimposed; abbreviations such as “BD”, “TDS”, and “OD” are used inconsistently; and numerical dosages are frequently compressed or partially overwritten. Studies across multiple healthcare systems have found that a significant proportion of prescriptions contain at least one item that is illegible to a lay reader, and in many cases to trained pharmacy staff as well [4]. The consequences range from incorrect drug selec-

tion and improper dosage administration to adverse drug reactions and, in severe cases, hospitalisation.

The second problem is the failure of existing recognition technologies. Conventional optical character recognition systems — including those employing deep learning architectures — are trained predominantly on printed typefaces and exhibit sharp accuracy degradation when applied to cursive, connected, or stylistically inconsistent handwriting. Medical prescription content compounds this difficulty through the use of Latin abbreviations, drug name truncations, and superimposed characters that bear little resemblance to standard typographic forms. Prescription images captured using mobile devices frequently suffer from variable lighting, motion blur, perspective distortion, and background noise — conditions that degrade recognition performance further in practical deployment.

The third problem extends beyond character recognition to semantic interpretation. Consider the string "Dolo 650 BD AC" — three words that a pharmacy technician reads instantly but that represent an entirely opaque instruction to most patients. Assembling that string into "Paracetamol 650 mg, twice daily, before meals" requires brand-to-generic drug mapping, abbreviation expansion, and knowledge of Indian dosage conventions. Generic OCR engines produce the string. They cannot produce the interpretation. This gap demands domain knowledge that standard text extraction systems do not possess.

The fourth problem is the absence of integrated medication management. Existing consumer health applications address either prescription digitisation or medication reminders, but no reviewed application provides both within a single workflow. Patients who wish to digitise their prescriptions must manually transcribe extracted text into a separate tracking application, introducing additional opportunities for error. Secure prescription storage with user-specific access control, chronological record retrieval, and automated reminders derived directly from extracted schedule data are missing from all reviewed consumer healthcare tools.

A further challenge is domain specificity. Indian prescriptions frequently contain locally prescribed brand names (e.g., Dolo 650, Pan-D, Glycomet), regional abbreviations (e.g., "AC", "PC"), and dosage notations (e.g., "1-0-1") that are poorly represented in the training data of generic OCR and language models. Systems developed for Western clinical settings do not generalise to these conditions — yet the Indian healthcare context is precisely where automated prescription interpretation is most needed.

Finally, the computational cost of AI-capable models poses a deployment challenge. Running deep learning models synchronously within a request-response cycle produces unacceptable client-facing latency. Most existing prototypes do not incorporate asynchronous processing architectures, limiting their viability in real-world, multi-user deployments.

MedMate addresses all six of these gaps by combining dedicated text detection, vision-language correction, and domain-aware structured extraction in a sequential pipeline that connects directly to a patient-facing medication management interface — eliminating every manual step between prescription receipt and reminder delivery.

1.3 Objectives

The primary objective of the MedMate project is to design, implement, and evaluate an AI-driven mobile healthcare system that transforms handwritten medical prescriptions into structured digital records and integrates the extracted information into a patient-centric medication management workflow.

The specific objectives of the project are enumerated below.

O-1: Accurate Prescription Interpretation through a Multi-Stage AI Pipeline: To develop a five-stage sequential artificial intelligence pipeline that preprocesses prescription images for real-world imaging conditions, detects word-level text regions, performs optical character recognition, corrects recognition errors using vision-language reasoning, and extracts structured medical entities — including

medicine names, generic equivalents, dosages, administration frequencies, and intake instructions — with target accuracies of at least 85% for medicine names, 90% for dosage values, and 85% for frequency identification on smartphone-captured handwritten prescriptions. Accuracy is defined as field-level exact or semantically equivalent match against manually annotated ground truth for each extracted medicine record.

O-2: Secure and Responsive System Architecture:

To design and implement a client–server architecture comprising a cross-platform mobile application and a backend API service, with token-based authentication, persistent storage of user profiles and prescription records, and asynchronous execution of computationally intensive AI tasks to ensure that the mobile client remains responsive throughout processing.

O-3: Structured Data Presentation and Prescription Management:

To present extracted prescription data to users in a clear, structured, and clinically informative format, and to maintain a searchable digital history of all processed prescriptions with user-specific access control, eliminating the need for manual transcription or reliance on fragmented tools.

O-4: Automated Medication Reminder and Adherence Support:

To enable users to configure personalised medication reminder schedules directly from extracted prescription data — including multiple daily doses, configurable time intervals, food-related instructions, and bedtime adjustments — with local device notifications incorporating a three-ring retry pattern to reduce missed-dose events, and the ability to record dose completion from notifications or the application dashboard.

O-5: Validation and Extensible Design:

To validate the system through unit, integration, system, and user acceptance testing across all functional and non-functional requirements, and to ensure that the

architecture supports future enhancements — such as multilingual prescription support, drug interaction checking, and integration with external healthcare systems — without requiring fundamental structural changes.

1.4 Scope of the Project

MedMate is scoped as a mobile-first, AI-driven healthcare application that covers the complete workflow from handwritten prescription image capture to structured medical data extraction and personalised medication adherence support. The system targets individual patients as primary users and operates within a secure client–server architecture with asynchronous AI processing.

The following functional areas are included within the current implementation:

- **User Management:** Secure user registration and authentication using token-based session management, along with per-user data isolation to ensure that prescription records are accessible only to the respective account owner.

- **Prescription Digitisation:** Upload of prescription images via the mobile application (device gallery selection; direct camera capture is identified as a planned near-term enhancement), with each prescription tracked through a defined status lifecycle from submission through processing to completion.

- **AI-Based Prescription Interpretation:** A five-stage pipeline that preprocesses images for real-world quality variations, detects word-level text regions using the CRAFT detector, performs optical character recognition via PaddleOCR, corrects errors through Qwen2-VL vision-language reasoning, and extracts structured medical entities — including medicine names, generic equivalents, dosages, frequencies, and intake instructions — via Qwen2.5-7B structured extraction.

- **Prescription History and Storage:** Persistent storage of all uploaded images and

extracted medical data in a relational database, with API-based retrieval enabling users to access their complete prescription history.

- **Medication Reminders and Adherence Tracking:** Configuration of personalised reminder schedules derived from extracted prescription data, with local device notifications supporting multiple daily doses, configurable time intervals, food-related instructions, and a three-ring retry notification pattern. Users can record dose completion directly from notifications or the application dashboard.

The system is optimised for single-page, English-language prescriptions within the Indian healthcare context, with recognition oriented toward commonly prescribed Indian brand names, abbreviations, and dosage conventions.

The following areas are explicitly outside the current scope:

- Multi-page prescription processing.
- Prescriptions in languages other than English.
- Direct camera capture within the mobile application (planned as a near-term enhancement; current implementation supports gallery selection only).
- Integration with hospital information systems, electronic health records, or pharmacy billing platforms.
- Real-time consultation with physicians or pharmacists.
- Drug interaction analysis or clinical decision support.

- Web-based interfaces; the system is mobile-only.

- Cloud-based auto-scaling or distributed deployment.

- Cross-device synchronisation of reminder configurations.

- Offline AI inference; an active backend connection is required for prescription processing, though reminders operate independently once configured.

The accuracy of extraction is dependent on the legibility of the input image and the resolution of the captured photograph. Severely degraded images may yield incomplete results. AI inference requires a GPU-equipped backend server; processing times on CPU-only hardware are not performance-guaranteed.

1.5 Limitations

The MedMate system, while designed to provide an effective end-to-end solution for prescription digitisation and medication management, operates within a defined set of limitations. These are documented to establish realistic expectations and to identify areas for future enhancement.

AI and Recognition Limitations: The system processes single-page, English-language prescriptions only. Multi-page documents and prescriptions written in regional or non-Latin scripts are not supported. Recognition accuracy is dependent on input image quality; severely blurred, low-resolution, or heavily occluded images may yield incomplete or incorrect extraction results. The AI models are optimised for commonly prescribed Indian brand names and standard medical abbreviations — uncommon drug names, rare shorthand, or highly stylised handwriting may not be reliably interpreted. The correction and extraction stages improve accuracy but do not guarantee clinical correctness in all cases.

Clinical Limitations: The system performs information extraction and structuring only. It does not provide medical diagnosis, drug interaction analysis, contraindication screening, or clinical decision support. Dose flags generated by the reasoning layer are based on predefined knowledge and do not account for patient-specific factors such as age, weight, allergies, or comorbidities. All extracted information should be verified by the user or a qualified healthcare professional before use.

Infrastructure and Performance Limitations: The AI pipeline requires a GPU-equipped backend server for acceptable processing times. CPU-only execution results in significantly longer inference and may not meet the performance targets defined in Section 3.5.1. All AI processing is asynchronous and server-dependent; the mobile client cannot perform extraction without an active internet connection. The current deployment uses a single task worker, meaning concurrent prescription uploads are processed sequentially rather than in parallel. Automated scaling, load balancing, and result caching are not implemented in the present version.

Functional Limitations: Users cannot manually edit or correct extracted prescription data after processing; any correction requires re-uploading the prescription image. Medication reminder configurations are stored locally on the device and are not synchronised across devices or backed up to the server. Prescription data cannot be exported in standard formats for sharing with healthcare providers. The system does not support real-time interaction with physicians or pharmacists. Direct camera capture within the mobile application is not yet implemented; users must select images from the device gallery.

Security Limitations: Authentication is implemented through token-based mechanisms with one-time password verification, but the system does not enforce role-based access control — all authenticated users hold identical privileges. Prescription images are stored without encryption at rest, and detailed audit logging of data access events is not implemented in the current version. Application secrets — including the Django secret key and the email service credentials used for OTP dispatch — are currently stored as plaintext values in the configuration file. This configuration is unsuitable for any

internet-facing deployment; these credentials must be migrated to environment-variable-based secrets management before the system is exposed to external users.

These limitations are acknowledged as areas for future improvement, as outlined in Section 7.2.

CHAPTER 2: Literature Review

2.1 Introduction

The literature relevant to MedMate spans five research domains: optical character recognition for handwritten and medical text, text detection in unstructured images, vision-language models for document understanding, large language models for clinical text processing, and mobile health reminder systems with asynchronous backend architectures. This chapter reviews prior work in each domain, identifies the limitations of existing approaches with respect to handwritten prescription digitisation, and presents the research gap that motivates the design decisions in MedMate.

2.2 Related Work

2.2.1 Optical Character Recognition for Medical Documents

The earliest computational approaches to prescription text extraction relied on general-purpose OCR engines. Tesseract, originally developed at Hewlett-Packard and subsequently maintained as an open-source project, became the standard for printed document recognition. Smith [1] documented Tesseract's architecture and noted that its recognition models are trained on typographic fonts, making the engine poorly suited to cursive or connected character forms. Subsequent evaluations on handwritten clinical documents confirmed this limitation: Krishnan et al. reported a character-level error rate exceeding 20% when applying Tesseract to uncontrolled handwritten clinical notes [5], attributable to the high intra-class variability of handwritten character forms that the engine was never trained to handle.

Deep learning OCR systems marked a substantial improvement. Du et al. [1] introduced PP-OCRv2, combining a Differentiable Binarisation network (DB-Net) for text detection with a Convolutional Recurrent Neural Network (CRNN) for sequence recognition.

PP-OCRv2 achieves competitive results on standard benchmarks and demonstrates particular strength on structured numerical content such as dosage quantities, tablet counts, and administration frequencies. However, its accuracy degrades on cursive medicine names where character spacing is irregular and letter forms deviate substantially from the training distribution.

A consistent finding across the OCR literature is that no single engine achieves satisfactory performance across the full range of prescription content. Numerical values are handled reliably by CRNN-based systems, but medicine names — frequently abbreviated or written in shorthand — require contextual reasoning that character-level recognition cannot provide. Augusto et al. [6] proposed a CNN-based segmentation approach that targeted drug name regions specifically, achieving improved localisation accuracy on printed prescription layouts, but the method did not extend to handwritten content and produced no structured clinical output beyond bounding box predictions.

2.2.2 Text Detection in Unstructured Images

Text localisation in complex, unstructured images has advanced substantially through pixel-level prediction frameworks. Baek et al. [2] proposed CRAFT (Character Region Awareness for Text Detection), which predicts two spatial confidence maps simultaneously: a character-region heatmap encoding per-pixel probabilities of individual character centres, and an affinity map encoding spatial linkage confidence between adjacent characters. Joint thresholding of these maps yields bounding polygons that are robust to curved, oriented, and irregularly spaced text — properties that are consistently present in handwritten prescriptions.

Competing detectors including EAST [3] and TextBoxes perform adequately on printed, horizontally aligned text but degrade on the irregular layouts produced by handwritten content. CRAFT's primary limitation in the medical domain is threshold calibration: default confidence settings tuned for printed text frequently produce sparse detections on the thinner, more variable strokes of handwritten characters. Effective deployment in a prescription digitisation context therefore requires domain-specific threshold adjustment

rather than out-of-the-box application.

2.2.3 Vision-Language Models for Document Understanding

Vision-language models (VLMs) represent a paradigm shift in document understanding by jointly encoding visual and textual information within a single transformer architecture. Models such as Donut [4] and Pix2Struct [5] treat document reading as a visual question-answering task, processing the entire document image in a single forward pass without an explicit OCR step. This design eliminates the error propagation that accumulates when OCR output is fed to a downstream language model.

Wang et al. [2] introduced Qwen2-VL, which employs a dynamic resolution mechanism that adaptively adjusts the number of visual tokens based on input image dimensions, preserving fine-grained spatial details critical for distinguishing handwritten character forms. The model demonstrated the ability to correct OCR errors by cross-referencing visual content against recognised text: an output such as "20g mg" is resolved to "200 mg" through contextual visual reasoning about digit spacing and character shape.

2.2.4 Large Language Models for Clinical Text Processing

Large language models (LLMs) have demonstrated strong capability in clinical natural language processing tasks. Singhal et al. [1] showed that large-scale language models encode substantial clinical pharmacological knowledge. Toma et al. [3] demonstrated that structured prompting improves extraction consistency. A growing body of work [6,7,8] also highlights the broader utility of mobile-assisted medication management through AI-driven tools.

2.2.5 Mobile Health Reminder Systems and Asynchronous Processing

Medication non-adherence is a well-documented contributor to treatment failure. Dayer et al. [2] conducted a systematic review of medication adherence applications. Heron and Smyth [3] studied notification strategies and found improved response rates.

From a systems architecture perspective, asynchronous processing is required. Celery with Redis is widely used for distributed task execution in Python-based systems [1].

2.3 Research Gap

Each of the five research threads reviewed in Section 2.2 has produced genuine progress in isolation. Better OCR engines exist. More robust text detectors exist. Clinical LLMs with strong pharmacological knowledge exist. Medication reminder applications with demonstrated adherence benefits exist. What does not exist — in any system reviewed for this work — is a single architecture that connects all five threads: one that takes a smartphone photograph of a handwritten prescription and produces a structured, locally-executed medication reminder schedule without any manual step in between.

The architectural gap. Existing prescription digitisation systems fall into one of two categories. The first applies a single OCR engine to the full prescription image, achieving acceptable accuracy on printed numerical dosages but failing on cursive medicine names where character spacing is irregular. The second uses a vision-language model as a standalone extractor, gaining contextual reasoning at the cost of numerical precision. As established by the accuracy patterns reviewed in Sections 2.2.1 and 2.2.3, CRNN-based systems and VLMs have complementary failure modes. To the best of the authors' knowledge, no reviewed work exploits both approaches in sequence — using crop-level OCR where it excels on numerical content, then applying vision-language correction on the ambiguous character regions where it fails. The complementarity is documented in the individual literatures; the joint exploitation within a single pipeline is not.

The environmental gap. Almost all published recognition models are evaluated on flatbed-scanned documents or printed text under controlled studio lighting. A patient photographing a prescription on a clinic counter does not produce flatbed-quality input: the image has perspective distortion from a slightly tilted phone, uneven illumination from an overhead light or nearby window, and motion blur from an unsteady hand. None of the reviewed systems incorporate a dedicated preprocessing stage designed for these specific degradation modes. This is not a minor academic limitation — it is the

difference between a system that functions in a controlled evaluation setting and one that functions in the field.

The integration gap. Every medication reminder application reviewed requires patients to type in their own medicine names, dosages, and schedules — the very information already written on the prescription they are holding. Every prescription digitisation system reviewed produces text or semi-structured output and stops there, leaving patients to manually transfer that output into a separate tracking application. These two functions have been studied in parallel for over a decade without being connected by any reviewed solution. MedMate closes this gap: the structured data extracted by the AI pipeline flows directly into the reminder scheduler, with no transcription step in between.

Indian prescriptions introduce a further dimension that generic systems consistently overlook. Brand names such as Dolo 650 and Pan-D, frequency notations such as "1-0-1", and regional abbreviations such as "AC" and "PC" appear in the majority of Indian outpatient prescriptions but are absent from the training distributions of OCR and language models built for Western clinical settings. This is precisely the context where automated prescription interpretation would provide the greatest benefit and where existing tools are least equipped to deliver it.

MedMate addresses all four of these gaps. The five-stage pipeline — CRAFT detection, PaddleOCR recognition, Qwen2-VL correction, and Qwen2.5 structured extraction — connects directly to a Flutter mobile application that schedules local dose reminders from the extracted data. The patient photographs the prescription; the system handles every subsequent step.

CHAPTER 3: Requirement Analysis and System Specification

3.1 Introduction

This chapter provides the complete requirements specification for the MedMate system. It presents a structured feasibility analysis across technical, operational, and economic dimensions; enumerates the hardware and software prerequisites; and formally states the functional requirements (FR-01 through FR-10) and non-functional requirements (NFR-01 through NFR-06) against which the implementation described in Chapters 4 and 5 is evaluated. A validation strategy mapping each requirement to a test level is presented in Section 3.6. The development methodology and sprint plan governing the construction of the system are documented in Section 4.2.

3.2 Feasibility Study

3.2.1 Technical Feasibility

The MedMate backend stack — Django 4.2, Django REST Framework 3.14, Celery 5.x, and Redis 6.x — is entirely open-source with no licensing cost. All four components have been in active production use for over five years and have well-documented deployment patterns for the exact architecture MedMate uses: a REST API server dispatching GPU-bound tasks to background workers through a Redis message queue. The Flutter 3.x SDK, maintained by Google, supports Android 6.0 (API Level 23) and above, covering the target device range without requiring proprietary mobile tooling.

All AI models in the pipeline are publicly available under permissive licences. The CRAFT text detector and Classic PaddleOCR (PP-OCRv3) are deployed locally on CPU and GPU respectively. The Qwen2-VL-2B correction model is loaded in 4-bit NF4 quantisation via the `bitsandbytes` library, consuming approximately 1,478 MiB of VRAM. Classic PaddleOCR operates on CPU, consuming zero VRAM. The Qwen2.5-

7B extraction model is accessed through the HuggingFace Inference Router API, eliminating the need to self-host a 7-billion-parameter model locally. Total local VRAM consumption is approximately 1,478 MiB, well within the 4,096 MiB capacity of the reference NVIDIA RTX 3050 used for development and testing.

The PostgreSQL 14+ database supports concurrent operations, referential integrity enforcement, and schema migration via Django's ORM. The modular three-application Django project structure (`users`, `prescriptions`, `ai_engine`) allows individual components to be updated independently. The system has been validated end-to-end on an NVIDIA RTX 3050 (4,096 MiB VRAM) running CUDA 13.0, confirming that the full pipeline executes within 41–46 seconds on accessible consumer hardware.

3.2.2 Operational Feasibility

MedMate is designed for patients with no technical background. The Flutter application follows Material Design 3 guidelines. All critical operations — prescription upload, viewing extracted data, and configuring reminders — are reachable within two navigation steps from the home dashboard. Input controls use time pickers, steppers, and toggles rather than free-text fields wherever possible, reducing the probability of user error during reminder configuration.

The asynchronous processing architecture keeps the application responsive during AI inference: the mobile client receives an immediate HTTP 201 acknowledgement on upload and polls the status endpoint every 20 seconds, displaying a progress indicator rather than blocking. The medication reminder system executes entirely on-device via `flutter_local_notifications` after initial configuration, functioning independently of network connectivity. This is important for users in areas with intermittent internet access.

Operationally, the system runs as a standard Linux web application. Database management is performed via pgAdmin or the Django admin interface; Celery worker health and task throughput are monitored via Flower; structured logs are emitted to stdout and

captured by the host process manager. Routine maintenance — model weight updates, database migrations, and log review — requires no specialised tooling beyond standard Python and PostgreSQL administration.

3.2.3 Economic Feasibility

All primary software dependencies — Django, Flutter, Celery, Redis, PostgreSQL, PaddleOCR, PyTorch, and Qwen model weights — are open-source and royalty-free. The HuggingFace Inference Router provides a free-tier API sufficient for development and low-volume testing of the Qwen2.5-7B extraction stage.

For the academic development phase, infrastructure cost is limited to the development machine (NVIDIA RTX 3050 laptop) already available to the project. A minimal production deployment on a cloud GPU instance — for example, one AWS `g4dn.xlarge` (4 vCPUs, 16 GB RAM, NVIDIA T4 16 GB VRAM) at approximately $0.53/hour, running during active use only — would cost under $130 per month at eight hours of daily operation. This is within the budget range of a small-scale academic deployment and can be reduced further by scheduling workers only during expected usage hours.

3.3 Software and Hardware Requirements

3.3.1 Hardware Requirements

Table 3.1 specifies the minimum hardware configuration below which the system is not guaranteed to operate, and the recommended configuration at which it has been validated to meet all performance targets.

3.3.2 Software Requirements

Table 3.2 lists the software dependencies and their minimum tested versions.

Table 3.1: Hardware Requirements

Component	**Specification**
Backend CPU	Intel Core i7 / AMD Ryzen 7 or equivalent (8+ cores)
Backend RAM	16 GB
GPU (Backend)	NVIDIA GPU, minimum 4 GB VRAM, CUDA 11.x compatible
Backend Storage	Minimum 50 GB available
Client Device RAM	Minimum 2 GB
Client Device Storage	Minimum 100 MB available
Client OS	Android 6.0 (API Level 23) or later

Table 3.2: Software Requirements

Component	**Version**
Python	3.10.x or later
Django	4.2.x (LTS)
Django REST Framework	3.14.x
Celery	5.x
Redis	6.x or later
PyTorch	2.x with CUDA 11.x
PaddlePaddle	2.6.x with CUDA
PaddleOCR	2.7.x
PostgreSQL	14 or later
Flutter SDK	3.x
Dart SDK	3.x

3.4 Functional Requirements

The following functional requirements define the observable behaviours that the MedMate system must exhibit. Each requirement is stated atomically — one condition, one behaviour, one testable outcome — consistent with IEEE 29148-2018. Requirements are identified by formal identifiers FR-01 through FR-10.

FR-01: User Registration. The system shall allow a new user to register by providing a unique username, a unique email address, and a password of at least eight characters. On successful registration, the system shall create a user account and dispatch a one-time password (OTP) to the registered email address. If the username or

email address already exists in the database, the system shall reject the request and return a field-level error response with HTTP status 400.

FR-02: OTP Verification and Account Activation. The system shall verify a submitted OTP against the stored value for the associated user account. A correct OTP submitted within ten minutes of generation shall activate the account and return a valid JWT access token and refresh token pair. An expired OTP, an incorrect OTP, or an OTP submitted after five failed attempts shall be rejected with a descriptive HTTP 400 error response. Each OTP shall be valid for a single successful verification only.

FR-03: Authenticated Login. The system shall authenticate a registered, active user who submits valid credentials (email address and password) by returning a JWT access token, a refresh token, and the user profile data in a single HTTP 200 response. A user whose account has not been OTP-verified (inactive) shall receive an HTTP 401 response. Invalid credentials shall receive an HTTP 401 response with a non-specific error message.

FR-04: Token Refresh and Logout. The system shall issue a new access token in exchange for a valid, non-blacklisted refresh token. On logout, the system shall blacklist the submitted refresh token, preventing its subsequent reuse. Rotation of a refresh token shall automatically blacklist the previous token.

FR-05: Password Change. The system shall allow an authenticated user to change their password by providing the current password and a new password of at least eight characters. An incorrect current password shall result in an HTTP 400 rejection. A successful change shall not invalidate existing active sessions.

FR-06: Prescription Image Upload — Validation. The system shall accept prescription

image uploads only in JPEG, PNG, or WebP format and only where the file size does not exceed 10 MB. MIME type shall be validated using the file's binary signature (magic bytes) rather than the file extension alone. Files that fail format or size validation shall be rejected with HTTP 400 and a descriptive error message identifying the specific constraint violated. Unauthenticated upload attempts shall be rejected with HTTP 401.

FR-07: Prescription Image Upload — Storage and Dispatch. Upon successful validation, the system shall store the image to the configured media directory, create a prescription record with status `pending`, and return HTTP 201 with the unique prescription identifier. The background AI processing task shall be enqueued only after the database transaction creating the prescription record is committed, to prevent the worker from reading a non-existent record.

FR-08: Five-Stage AI Pipeline Execution. The background task shall execute a five-stage AI pipeline in the following fixed sequence:

(1) **Image Preprocessing** (OpenCV): deskew, denoise, CLAHE contrast enhancement, and resize to a minimum of 1,000 pixels on the longer axis.

(2) **Text Detection** (CRAFT): generate character region heatmaps and affinity maps; merge proximate bounding boxes into line-level regions; extract cropped patches.

(3) **OCR Extraction** (Classic PaddleOCR PP-OCRv3): recognise text from cropped patches on CPU; filter detections by a minimum confidence of 0.3.

(4) **Vision-Language Correction** (Qwen2-VL-2B INT4): cross-reference OCR output against the original image to correct character-level errors.

(5) **Structured Extraction** (Qwen2.5-7B-Instruct): resolve brand names, expand abbreviations, and produce a validated JSON object containing nine per-medicine fields and a pipeline confidence rating.

The prescription status shall be updated to `processing` when the task begins execution. Any unrecoverable pipeline failure shall set the status to `failed`.

FR-09: Medicine Data Persistence and Status Update. The system shall parse the structured pipeline output and create one database record per identified medicine linked to the parent prescription. The nine stored fields per medicine are: `name`, `generic_name`, `dosage`, `frequency`, `duration`, `instructions`, `type`, `dose_flag`, and `dose_flag_reason`. Fields absent from the pipeline output shall be stored with the sentinel value `Not specified`; no null values shall be written to medicine records. The prescription status shall be updated atomically to `processed` (one or more medicines extracted), `processed_empty` (pipeline completed successfully but no medicines identified), or `failed` (pipeline error), based on the outcome.

FR-10: Prescription Retrieval. The system shall provide two authenticated API endpoints: one returning a paginated list of the requesting user's prescriptions ordered by creation date descending, and one returning the full detail of a single prescription including all extracted medicine records. Both endpoints shall enforce per-user data isolation — a user shall not be able to retrieve prescriptions belonging to any other account. Requests from unauthenticated clients shall receive HTTP 401.

3.5 Non-Functional Requirements

3.5.1 NFR-01: Performance

The backend API shall return an HTTP acknowledgement response to a prescription upload request within 2 seconds under normal operating conditions, independent of AI processing duration. The five-stage AI pipeline shall complete processing of a single-page prescription image within 30 seconds on the reference GPU hardware (NVIDIA

RTX 3050, 4 GB VRAM). Prescription list and detail endpoints shall return responses within 1 second for users with up to 100 prescription records. The mobile application shall render screens within 1 second of receiving an API response.

The current deployment supports a single concurrent AI processing task; concurrent prescription uploads are queued and processed sequentially. This is an accepted constraint of the present single-worker configuration, as documented in Section 1.5.

3.5.2 NFR-02: Reliability

The background task worker shall implement automatic retry logic for transient failures (`OSError`, `RuntimeError`), with a maximum of three retry attempts using exponential backoff with an initial interval of five seconds. Permanent failures (`FileNotFoundError`, `Prescription.DoesNotExist`) shall result in immediate task termination with the prescription status set to `failed`, without triggering the retry cycle.

The system shall ensure that a prescription is processed at most once, even if the same task is enqueued multiple times due to message broker redelivery. This is enforced by a pessimistic row-level lock (`select_for_update`) that succeeds only when the prescription status is `pending`; subsequent executions exit cleanly without duplicating work.

Fields absent from the pipeline output shall be stored with the sentinel value `Not specified`, ensuring that no null values are written to medicine records and that referential integrity is maintained across all nine per-medicine fields. The mobile application shall continue to display scheduled reminders and allow dose marking when the device is offline; reminder delivery does not depend on network connectivity.

3.5.3 NFR-03: Security

All communication between the mobile application and the backend shall be conducted over HTTPS in production deployments. User passwords shall be stored exclusively as PBKDF2-SHA256 salted cryptographic hashes and shall never appear in plaintext in

application logs, API responses, or database records.

JWT access tokens shall have a configurable expiry period not exceeding 30 minutes. Refresh tokens shall be invalidated on logout and on rotation, preventing reuse of superseded tokens. The upload endpoint shall validate file type by binary magic-byte inspection and reject files exceeding 10 MB, mitigating upload-based attack vectors.

Each prescription record shall be accessible only to the authenticated user who created it, enforced at the ORM query layer using `request.user` filtering on all prescription endpoints. All application secrets — including the Django `SECRET_KEY`, database credentials, and email service credentials — shall be stored in environment variables and shall not be hardcoded in source code or committed to version control.

3.5.4 NFR-04: Scalability

The system architecture shall support horizontal scaling of the AI processing layer by adding additional Celery worker processes without requiring changes to application logic or database schema. Each worker shall load AI model weights into GPU memory at initialisation and reuse the loaded models for all subsequent tasks in that worker's lifetime, avoiding repeated weight loading overhead.

The PostgreSQL database shall support concurrent read and write operations from multiple application and worker processes. The Redis message broker shall support task routing across multiple named queues to enable future specialisation of worker pools by task type.

3.5.5 NFR-05: Usability

The mobile application shall be operable by first-time users without formal training or a tutorial. Core workflows — prescription upload, viewing extracted data, and configuring reminders — shall be completable within two navigation steps from the home dashboard. Error messages shall be presented in non-technical language and shall include specific guidance for corrective action where applicable (e.g., "File size exceeds

10 MB. Please compress or re-capture the image.”).

The application shall poll the prescription status endpoint at 20-second intervals following upload submission and shall update the displayed status within one polling cycle of the backend status change. A visible progress indicator shall be displayed throughout the polling period.

3.5.6 NFR-06: Maintainability

The backend system shall follow a modular three-application Django architecture (`users`, `prescriptions`, `ai_engine`) with clear responsibility boundaries, enabling independent updates to any module without affecting the others. Each stage of the five-stage AI pipeline shall be implemented as a discrete Python module with a documented input and output interface, allowing individual model substitution without modifying the pipeline orchestrator.

The Flutter mobile application shall separate API communication (`ApiService`), notification scheduling (`NotificationService`), and local data persistence (`ReminderStorage`) into distinct service classes. All backend components shall emit structured log records with severity levels (`DEBUG`, `INFO`, `WARNING`, `ERROR`) to support operational monitoring and post-incident debugging.

3.6 Validation

This section defines the validation strategy confirming that the MedMate system satisfies the functional requirements specified in Section 3.4 (FR-01 through FR-10) and the non-functional requirements specified in Section 3.5 (NFR-01 through NFR-06). Validation is conducted across four levels corresponding to the V-model testing approach applied in Chapter 6.

- **Requirement Traceability.** Each functional requirement (FR-01 through FR-10) is mapped to one or more test cases in Chapter 6, ensuring that every specified behaviour has a corresponding verification procedure. Non-functional requirements

(NFR-01 through NFR-06) are validated through dedicated performance, security, and reliability test cases documented in Sections 6.2 through 6.4.

- **Unit Testing.** Individual backend components are tested in isolation using Django's `TestCase` framework. Components under unit test include: user registration serializer and OTP verification logic (FR-01, FR-02), prescription image MIME-type and size validation (FR-06), AI pipeline stage functions including preprocessing normalisation, CRAFT detection output format, PaddleOCR confidence filtering, and sentinel value substitution (FR-08, FR-09), and the Celery task idempotency guard (`select_for_update` lock behaviour) (NFR-02).

- **Integration Testing.** The interaction between the Django API, Redis message broker, Celery worker, and AI pipeline is validated against a running Redis instance and a dedicated test database. Integration tests cover: transaction-committed task dispatch after `on_commit` fires (FR-07), the complete `pending` → `processing` → `processed` status lifecycle (FR-09), pipeline behaviour on blank and corrupted input images yielding `processed_empty` and `failed` states (FR-09), and sentinel value substitution for absent medicine fields ensuring no null database writes (NFR-02, FR-09).

- **System Testing.** End-to-end user journeys are validated on a physical Android device connected to the development backend. Test scenarios include: complete registration and OTP verification (FR-01, FR-02), prescription upload through to medicine display (FR-06 through FR-10), reminder configuration and three-ring notification delivery with dose marking, daily dashboard reset, and logout with session invalidation (FR-04). All five system test scenarios are documented in Section 6.3.

- **User Acceptance Testing (UAT).** Eight representative users complete the full workflow from account registration through prescription upload to reminder delivery,

using printed and handwritten prescription samples, without assistance. Usability is rated on five dimensions using a five-point Likert scale. UAT results are reported in Section 6.4.

- **Performance Validation.** API response times are measured against the targets defined in NFR-01: upload acknowledgement within 2 seconds, five-stage AI pipeline completion within 30 seconds on the reference GPU, and list endpoint response within 1 second for a user with up to 100 prescription records. Pipeline timing results are reported in Section 5.4.1 (Table 5.1).

- **Security Validation.** Authentication boundary conditions are validated through dedicated test cases: expired access token rejection (NFR-03), invalid OTP handling including attempt-limit enforcement (FR-02), upload MIME-type spoofing via extension mismatch (FR-06), and cross-user prescription access attempts returning HTTP 403 (FR-10). SQL injection vectors are tested on all user-controlled string input fields.

CHAPTER 4: System Design

4.1 Introduction

This chapter presents the complete system design of MedMate. Section 4.2 describes the system architecture and its five principal components. Section 4.3 presents the UML diagrams that capture the structural and behavioural aspects of the system. Section 4.4 describes the user interface design of the Flutter mobile application. Section 4.5 documents the database schema and entity relationships. The design decisions in this chapter are grounded in the requirements established in Chapter 3 and informed by the literature surveyed in Chapter 2.

MedMate is built as a three-tier system: a Flutter mobile client, a Django REST backend, and a PostgreSQL database. A Celery worker connected to the backend via a Redis message broker handles AI inference asynchronously, separate from the HTTP request–response cycle. This separation exists because the five-stage AI pipeline takes 41–46 seconds on the reference hardware, which exceeds any reasonable HTTP timeout. The mobile client receives an immediate HTTP 201 acknowledgement on upload and polls for results at 20-second intervals until a terminal processing status is detected.

The five pipeline stages that transform a raw prescription image into structured medicine data are:

1. **Image Preprocessing** (OpenCV, PIL) — deskew, denoise, CLAHE contrast enhancement, and resize to a minimum of 1,000 pixels on the longer axis.

2. **Text Detection** (CRAFT) — character region heatmap and affinity map generation; bounding box merging; cropped word patch extraction.

3. **OCR Recognition** (Classic PaddleOCR PP-OCRv3, CPU) — text transcription from cropped patches with confidence filtering at threshold 0.3.

4. **Vision-Language Correction** (Qwen2-VL-2B, INT4 NF4) — character-level OCR error correction by cross-referencing the recognised text against the original prescription image.

5. **Structured Extraction** (Qwen2.5-7B-Instruct) — brand name resolution, Latin abbreviation expansion, and structured JSON output with clinical dose validation flags.

4.2 System Architecture

4.2.1 Architectural Overview

MedMate is built as a three-tier system: a Flutter mobile client, a Django REST backend, and a PostgreSQL database. A fourth component — a Celery worker connected to the backend via a Redis message broker — handles AI inference separately from the HTTP layer. Three design principles motivated this decomposition. First, the sequential AI pipeline introduces latency that is incompatible with synchronous API timeout constraints, requiring asynchronous decoupling. Second, separating concerns across tiers allows each component to be developed, tested, and replaced independently without cross-layer coupling. Third, a language-agnostic REST interface ensures that the backend can serve client platforms beyond the current Flutter application if future extensions are needed.

4.2.2 System Components

The MedMate system consists of five logically distinct components, described in the following subsections.

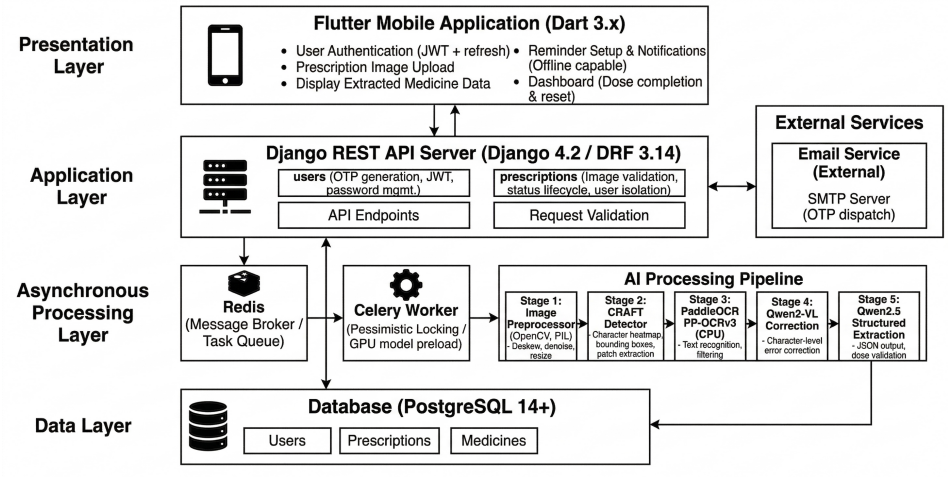


Figure 4.1: MedMate System Architecture

Presentation Layer — Flutter Mobile Application

The mobile client is built with Flutter 3.x (Dart 3.x), targeting Android 6.0 (API Level 23) and above. It handles six functional areas: JWT-based authentication with automatic token refresh on HTTP 401; device gallery image selection and multipart upload with 20-second status polling; structured display of extracted medicine data including dosage, frequency, and intake instructions; local reminder configuration persisted via `shared_preferences`; three-ring escalating dose notifications via `flutter_local_notifications`; and a daily medication dashboard with dose completion tracking and automatic midnight reset. Once reminders are configured, notification delivery operates entirely offline without requiring network connectivity.

Application Layer — Django REST API

The backend is implemented with Django 4.2 and Django REST Framework 3.14, structured into three Django applications. The `users` application handles the complete authentication lifecycle: OTP generation and email dispatch, JWT issuance via SimpleJWT, token rotation and blacklisting, and password management. The `prescriptions` application manages image upload with MIME-type and size validation, prescription status lifecycle tracking, and paginated retrieval with per-user data isolation enforced at the ORM level. The `ai_engine` application encapsulates the five-stage pipeline and is invoked exclusively through the Celery task worker. Two constraints govern the backend throughout: all AI tasks are dispatched using `transaction.on_commit()` so that the

Celery worker never reads a prescription record before it is committed to the database, and all querysets are filtered by `request.user` to enforce per-user data isolation.

Table 4.1: Backend Modules and Responsibilities

Module	**Responsibility**
`users`	Registration, OTP generation and verification, JWT issuance, token refresh and blacklisting, logout, password management
`prescriptions`	Image upload, MIME-type and size validation, status lifecycle tracking (`pending` → `processed`), paginated retrieval APIs
`ai_engine`	Encapsulation of the five-stage AI pipeline; invoked exclusively via the Celery task worker

Asynchronous Processing Subsystem — Celery and Redis

Celery 5.x with Redis 6.x as the message broker handles all prescription processing tasks asynchronously. The single task `process_prescription_ai(prescription_id)` encapsulates the complete pipeline execution. A pessimistic row-level lock via `select_for_update()` provides idempotency: a task that finds the prescription status is no longer `pending` exits cleanly without duplicating work, even if the same task is enqueued twice due to broker redelivery. AI models are preloaded into GPU memory at worker startup via the `worker_init` signal, avoiding repeated weight loading across tasks. Transient failures trigger automatic retries with a maximum of three attempts and exponential backoff starting at five seconds.

AI Processing Subsystem — Five-Stage Pipeline

The AI engine executes five sequential stages, each implemented as an independent Python module. Table 4.2 summarises the stages, their implementing modules, and their functions.

Data Layer — PostgreSQL and File Storage

PostgreSQL 14+ stores all structured data across three entity tables: users, prescriptions, and medicine records. Prescription images are written to `media/prescriptions/` on the backend server; direct file access is restricted and all image retrieval is mediated through authenticated API endpoints. Redis serves exclusively as the Celery message

Table 4.2: AI Pipeline Stages

Stage	Module	Model / Tool	Function
1	`image_preprocessor.py`	OpenCV, PIL	Deskew, denoise, CLAHE contrast enhancement, resize to $\geq$1,000 px on longer axis
2	`craft_detector.py`	CRAFT	Character heatmap and affinity map generation; bounding box merging; cropped patch extraction
3	`paddle_ocr.py`	PaddleOCR PP-OCRv3 (CPU)	Text recognition from cropped patches; confidence filtering at threshold 0.3
4	`qwen2_correction.py`	Qwen2-VL-2B (INT4 NF4)	Character-level OCR error correction via vision-language cross-referencing
5	`qwen_instruct.py`	Qwen2.5-7B-Instruct	Brand name resolution, abbreviation expansion, structured JSON extraction with dose validation

broker and holds no persistent application data.

4.3 UML Diagrams

Four UML diagram types are used to document MedMate's structure and behaviour. The **use case diagram** identifies the system boundary and the interactions between the two principal actors — the patient and the Celery worker — providing a complete map of user-visible system functionality. The **class diagram** documents the four Django model classes, their attributes, methods, and relationships, serving as the authoritative reference for the database schema in Section 4.5. The **sequence diagram** captures the prescription upload and AI processing workflow across six lifelines, making the `transaction.on_commit()` dispatch pattern and the asynchronous polling loop explicit in a way that prose alone cannot. The **activity diagram** shows the complete end-to-end user journey from application launch through daily reminder use, documenting all decision points and parallel flows.

4.3.1 Use Case Diagram

The use case diagram in Figure 4.2 identifies the two principal actors in the MedMate system. The **Patient** actor interacts directly with the mobile application and initiates all user-facing operations. The **Celery Worker** and **AI Engine** are system-side actors that execute prescription processing autonomously after the patient triggers an upload.

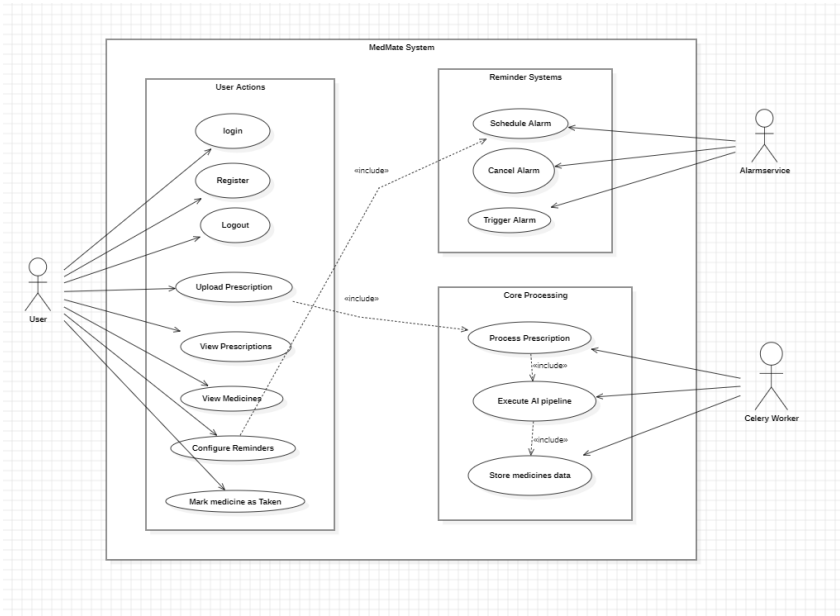


Figure 4.2: MedMate Use Case Diagram

4.3.2 Class Diagram

The class diagram in Figure **??** shows the four principal model classes of the MedMate backend and their relationships. The `User` class is Django's built-in `auth.User` model, providing `id`, `username`, `email`, `password`, and `is_active` attributes. The `EmailOTP` class has a OneToOne relationship with `User` and carries `otp`, `created_at`, `attempts`, and `verified` attributes, along with the `is_valid()`, `is_expired()`, and `generate_otp()` methods. The `Prescription` class has a ForeignKey relationship to `User` (one user, many prescriptions) and carries `image`, `status`, and `created_at` attributes. The `Medicine` class has a ForeignKey relationship to `Prescription` (one prescription, many medicines) and carries `name`, `generic_name`, `dosage`, `frequency`, `duration`, `instructions`, `type`, `dose_flag`, and `dose_flag_reason` attributes.

4.3.3 Sequence Diagram

The sequence diagram in Figure 4.3 captures the prescription upload and AI processing workflow across six lifelines: the Flutter mobile application, the Django REST API, the Redis message broker, the Celery worker, the AI engine, and the PostgreSQL database. The workflow begins when the patient submits a multipart POST request. The Django API validates the file, writes the prescription record with status `pending`, and returns HTTP 201 immediately. After the database transaction commits, the API enqueues the Celery task via Redis. The worker acquires a row-level lock, updates the status to `processing`, and invokes the five-stage pipeline. On successful completion, the medicine records are written and the status is updated to `processed`. The mobile client, polling every 20 seconds, detects the terminal status and navigates to the Medicines screen.

4.3.4 Activity Diagram

The activity diagram in Figure 4.4 shows the complete end-to-end user workflow, from the moment the patient opens the application through daily medication dashboard use. The diagram documents three primary decision points: account verification status at login, prescription processing outcome (processed, empty, or failed), and daily dose completion tracking. The reminder delivery branch operates independently of network connectivity once the medication schedule has been configured.

4.4 Database Design

4.4.1 Entity-Relationship Diagram

The MedMate data model comprises four principal entities shown in Figure **??**. The `User` entity stores account credentials and activation state. The `EmailOTP` entity has a mandatory one-to-one relationship with `User`, representing the OTP record for each account's verification lifecycle. The `Prescription` entity has a many-to-one relationship with `User`: one user may have many prescriptions, but each prescription belongs to exactly one user. The `Medicine` entity has a many-to-one relationship with `Prescription`: each prescription may yield many extracted medicine records, and each

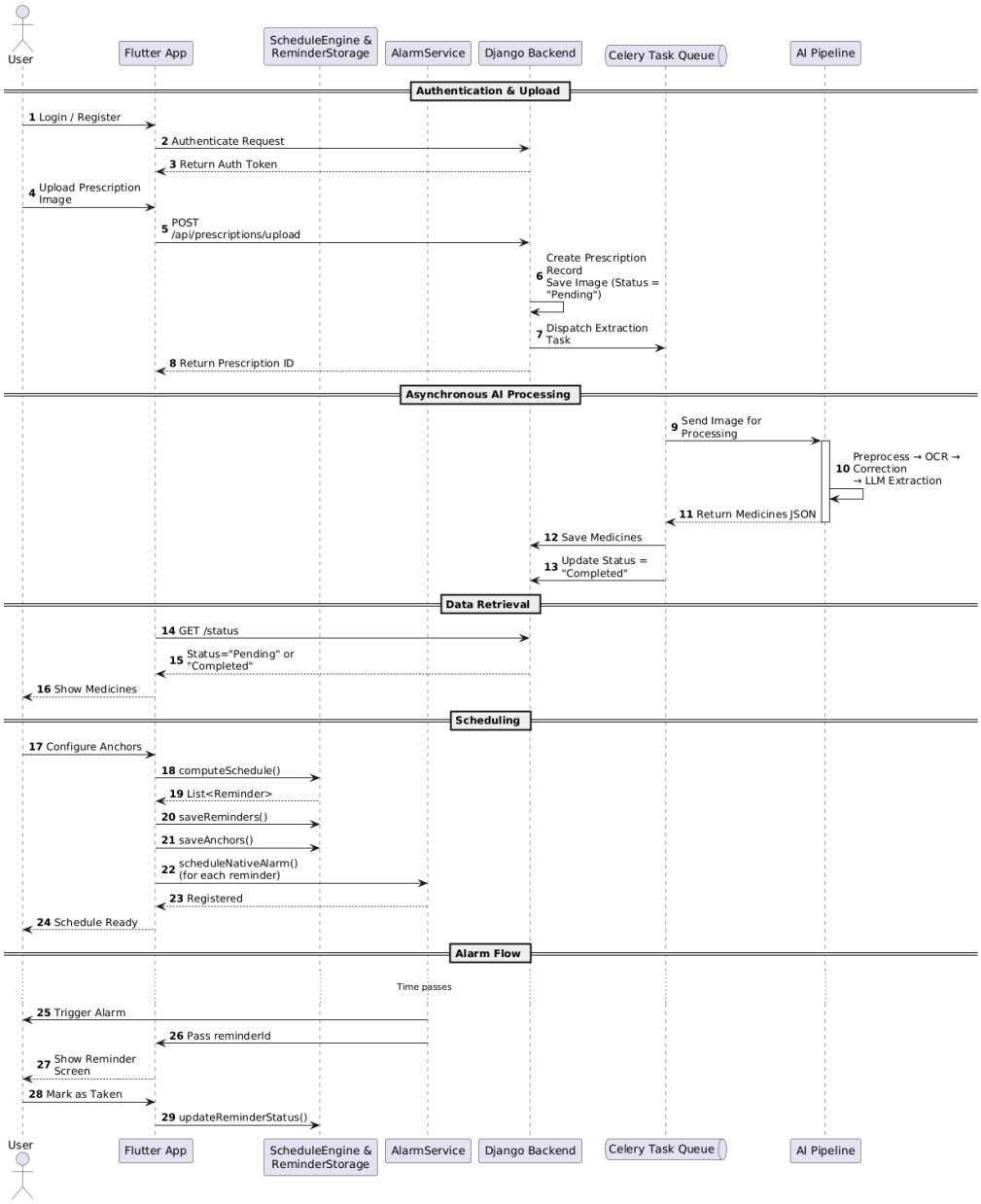


Figure 4.3: Sequence Diagram — Prescription Upload and AI Processing

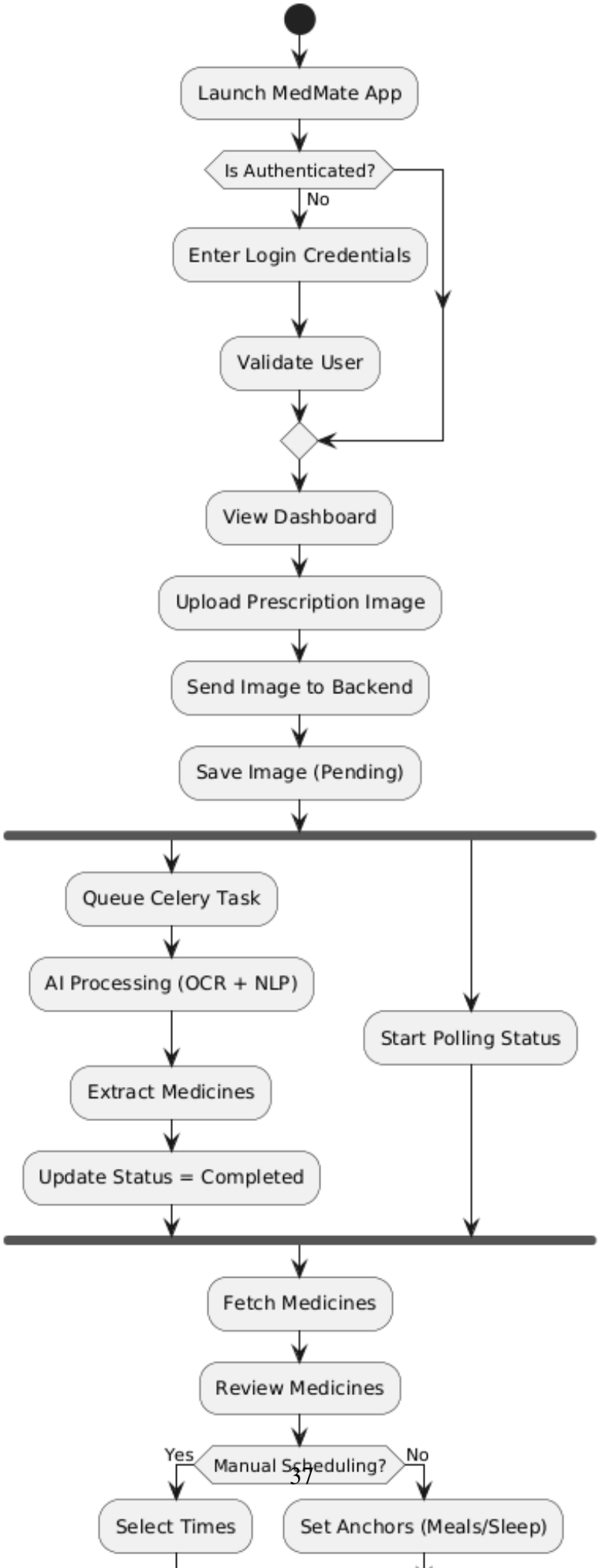
medicine record belongs to exactly one prescription. Deletion of a `User` cascades to all associated `Prescription` records; deletion of a `Prescription` cascades to all associated `Medicine` records.

4.4.2 Table Schemas

Table 4.3 documents the complete schema for all four database tables, including field names, data types, and constraints.

Table 4.3: Database Table Schemas

Table	Field	Type	Notes
`auth_user`	id	BigInt PK	Django default
	username	Varchar(150)	Unique
	email	Varchar(254)	Unique
	password	Varchar(128)	PBKDF2-SHA256 hash
	is_active	Boolean	False until OTP verified
`users_emailotp`	id	BigInt PK	
	user_id	FK → User	OneToOne, CASCADE
	otp	Varchar(6)	6-digit numeric
	created_at	DateTime	Auto on create
	attempts	Integer	Default 0, max 5
	verified	Boolean	Default False
`prescriptions_prescription`	id	BigInt PK	
	user_id	FK → User	CASCADE on delete
	image	ImageField	Path in `media/`
	status	Varchar(20)	`pending`, `processing`, `processed`, `processed_empty`, `failed`
	created_at	DateTime	Auto
`prescriptions_medicine`	id	BigInt PK	
	prescription_id	FK → Prescription	CASCADE
	name	Varchar(255)	
	generic_name	Varchar(255)	
	dosage	Varchar(100)	
	frequency	Varchar(100)	
	duration	Varchar(100)	
	instructions	Varchar(255)	
	type	Varchar(100)	
	dose_flag	Varchar(20)	OK / LOW / HIGH / VERIFY
	dose_flag_reason	Varchar(255)	


Launch MedMate App
Is Authenticated?
No
Enter Login Credentials
Validate User
View Dashboard
Upload Prescription Image
Send Image to Backend
Save Image (Pending)
Queue Celery Task
AI Processing (OCR + NLP)
Extract Medicines
Update Status = Completed
Start Polling Status
Fetch Medicines
Review Medicines
Yes
Manual Scheduling?
No
Select Times
Set Anchors (Meals/Sleep)

CHAPTER 5: Implementation, Discussions and Results

5.1 Introduction

The implementation phase transformed MedMate from a conceptual system design into a fully operational prescription digitisation and medication management application. This phase focused on integrating a five-stage AI pipeline — comprising CRAFT text detection, PaddleOCR recognition, Qwen2-VL visual correction, and Qwen2.5-7B structured extraction — with a Django REST backend, PostgreSQL database, and Flutter mobile application. By adopting the Agile Scrum methodology, core modules were developed incrementally across six sprints, validated through integration testing at each stage, and composed into a cohesive end-to-end system.

The implementation was carried out on a consumer-grade development machine equipped with an NVIDIA GeForce RTX 3050 Laptop GPU (4096 MiB VRAM), running Windows 10 (Build 26200), Python 3.10.11, and CUDA 13.0.

5.2 Method of Implementation

The MedMate system was implemented using an Agile Scrum-based iterative and incremental development lifecycle. Development was organised across six sprints spanning eleven weeks of active implementation effort.

5.2.1 Sprint 0 — Environment Setup and Requirements Finalisation

Technology Stack: Python 3.10.11, Django 4.x, PostgreSQL 18, Redis 7.x, Flutter SDK 3.x, CUDA 13.0, NVIDIA RTX 3050 (4096 MiB VRAM).

Implementation: The development environment was configured with all backend and frontend dependencies. The Django project structure was initialised with two registered

applications — `apps/users/` and `apps/prescriptions/` — alongside the `apps/ai_engine/` module hierarchy for pipeline components. PostgreSQL was configured as the primary database with the following Django settings:

```
# settings.py
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'medmate_db',
        'USER': 'postgres',
        'PASSWORD': os.environ.get('DB_PASSWORD'),
        'HOST': 'localhost',
        'PORT': '5432',
    }
}
CELERY_BROKER_URL = 'redis://127.0.0.1:6379/0'
```

Outcome: A fully configured development environment with project repository, initial database schema, and verified Django–PostgreSQL–Redis connectivity.

5.2.2 Sprint 1 — User Management and Authentication

Technology Stack: Django REST Framework, SimpleJWT, `python-magic`.

Implementation: JWT-based authentication was configured with 30-minute access tokens and 30-day refresh tokens with rotation and blacklisting enabled.

```
# serializers.py
class UserRegistrationSerializer(serializers.ModelSerializer):
    def create(self, validated_data):
        validated_data.pop('password2')
        user = User.objects.create_user(
```

```
            username=validated_data['username'],
            email=validated_data['email'],
            password=validated_data['password'],
            is_active=True,
        )
        return user

# views.py
class LoginView(APIView):
    permission_classes = [permissions.AllowAny]

    def post(self, request):
        serializer = EmailLoginSerializer(data=request.data)
        if not serializer.is_valid():
            return Response(serializer.errors,
                            status=status.HTTP_400_BAD_REQUEST)
        user = serializer.validated_data["user"]
        refresh = RefreshToken.for_user(user)
        return Response({
            "access":  str(refresh.access_token),
            "refresh": str(refresh),
            "user":    UserProfileSerializer(user).data,
        })
```

Outcome: Fully functional user registration, email-based login, JWT token issuance, token refresh, and logout with refresh token blacklisting. Flutter authentication screens completed and verified.

5.2.3 Sprint 2 — Prescription Ingestion and Status Tracking

Technology Stack: Django REST Framework, Celery 5.6.2, Redis, `python-magic`, Flutter `http` package.

Implementation: The prescription upload endpoint was implemented to accept multipart form data, validate MIME type using `python-magic`, enforce a 10 MB file size limit, and persist the image to `media/prescriptions/`. The Celery task is dispatched via `transaction.on_commit()`.

```
# views.py
class PrescriptionUploadView(generics.CreateAPIView):
    permission_classes = [IsAuthenticated]

    def perform_create(self, serializer):
        prescription = serializer.save(user=self.request.user)
        transaction.on_commit(
            lambda: process_prescription_ai.delay(
                prescription.id)
        )

# tasks.py
@shared_task(bind=True,
             autoretry_for=(OSError, RuntimeError),
             retry_backoff=5,
             retry_kwargs={"max_retries": 3})
def process_prescription_ai(self, prescription_id: int):
    with transaction.atomic():
        prescription = (
            Prescription.objects
            .select_for_update()
            .get(id=prescription_id, status="pending")
        )
        prescription.status = "processing"
        prescription.save(update_fields=["status"])
```

Outcome: Verified end-to-end prescription upload with asynchronous task dispatch and

status lifecycle management.

5.2.4 Sprint 3 — AI Pipeline Integration

Technology Stack: `craft-text-detector`, `paddleocr==2.7.3`, `paddlepaddle==2.6.2`, `transformers`, `bitsandbytes`, HuggingFace Inference Router, OpenAI Python client.

Implementation: The five-stage pipeline was implemented as a sequential chain orchestrated by `MedMatePipeline.process()`.

```
# pipeline.py
class MedMatePipeline:
    def process(self, image_path: str) -> dict:
        from apps.ai_engine.utils.image_preprocessor \
            import preprocess
        from apps.ai_engine.ocr.craft_detector \
            import detect_regions
        from apps.ai_engine.ocr.paddle_ocr \
            import extract_text
        from apps.ai_engine.vision_reasoner.qwen2 \
            import correct_ocr

        image       = preprocess(image_path)        # Stage 1
        regions     = detect_regions(image)         # Stage 2
        paddle_text = extract_text(regions)         # Stage 3
        corrected   = correct_ocr(image,
                                  paddle_text)      # Stage 4
        result      = self.reasoner.reason(
                          corrected)               # Stage 5
        return result
```

Outcome: Full five-stage pipeline operational. Prescription #1 yielded 2 medicines

with correct generic name resolution and dose flag activation. All nine medicine fields confirmed non-null in PostgreSQL across all extractions.

5.2.5 Sprint 4 — Mobile Reminder System and Dashboard

Technology Stack: Flutter, `flutter_local_notifications`, `shared_preferences`, `timezone`.

Implementation: A three-ring notification pattern fires the primary alert at the scheduled time with two follow-up rings at +2 and +4 minutes respectively.

```
// notification_service.dart
Future<void> scheduleThreeRingReminder({
  required int baseId,
  required String medicineName,
  required DateTime scheduledTime,
}) async {
  for (int ring = 0; ring < 3; ring++) {
    final ringTime = scheduledTime.add(
        Duration(minutes: ring * 2));
    await _notifications.zonedSchedule(
      baseId + ring,
      'Medicine Reminder',
      '$medicineName - Dose due',
      tz.TZDateTime.from(ringTime, tz.local),
      _notifDetails,
      androidScheduleMode:
          AndroidScheduleMode.exactAllowWhileIdle,
    );
  }
}
```

Outcome: Verified dose reminder scheduling with three-ring retry pattern, correct cancellation on "Mark Taken", and daily taken-status reset.

5.2.6 Sprint 5 — Testing, Documentation, and Deployment

Technology Stack: Django test client, Flutter device testing, Postman.

Implementation: Integration testing verified the complete request lifecycle. Performance testing confirmed the pipeline timing reduction from 333–367 seconds under the original implementation to 41–46 seconds, representing an approximately 8× end-to-end improvement.

Outcome: All tested workflows verified operational. System deployed to target hardware with verified end-to-end prescription digitisation and reminder delivery functionality.

5.3 AI Pipeline Implementation

5.3.1 Pipeline Overview

The MedMate AI pipeline accepts a raw prescription image and produces a structured JSON object containing extracted medicine data. The pipeline is implemented as a five-stage sequential chain under `apps/ai_engine/`, orchestrated by a central `MedMatePipeline` class.

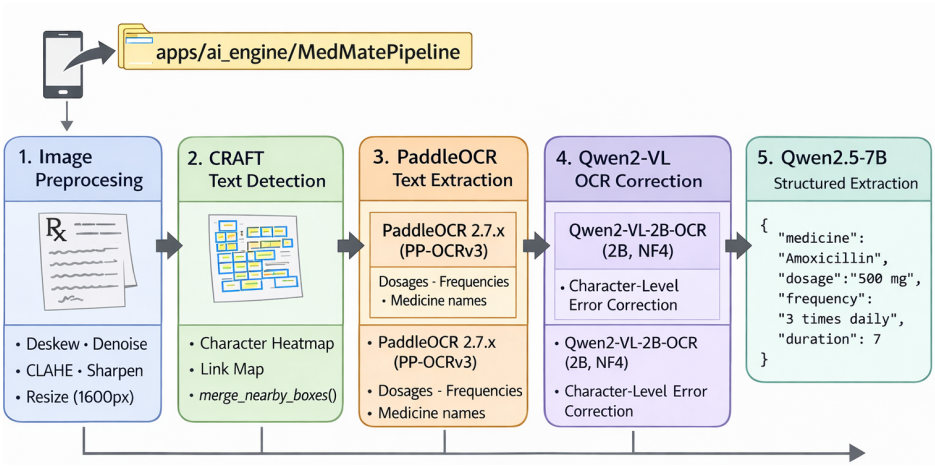


Figure 5.1: MedMate Five-Stage AI Pipeline Data Flow

5.3.2 Stage 1 — Image Preprocessing

The preprocessing module applies six sequential OpenCV operations to normalise input image quality. Images below 1000 pixels are upscaled to a 1600-pixel target using bicubic interpolation. Rotational misalignment is detected via Hough line transform and corrected. Non-local means denoising suppresses background noise. CLAHE improves local contrast. An unsharp mask is applied as a final sharpening step.

5.3.3 Stage 2 — CRAFT Text Detection

CRAFT was integrated using the `craft-text-detector` package. The detector produces two spatial confidence maps: a character region heatmap and an affinity map. A custom `merge_nearby_boxes()` post-processing function groups spatially proximate bounding boxes into coherent line-level text regions.

5.3.4 Stage 3 — PaddleOCR Text Extraction

Classic PaddleOCR 2.7.x employing the PP-OCRv3 recognition backbone operates entirely on CPU. The recogniser employs a DB-Net detection backbone combined with a CRNN sequence recognition model, with confidence filtering at threshold 0.3.

5.3.5 Stage 4 — Qwen2-VL OCR Correction

The `JackChew/Qwen2-VL-2B-OCR` model is loaded in 4-bit NF4 quantisation consuming approximately 1,478 MiB of VRAM. Its function is strictly character-level error correction: the model receives PaddleOCR output alongside the preprocessed image and corrects misread characters through visual cross-referencing.

5.3.6 Stage 5 — Qwen2.5-7B Structured Extraction

`Qwen/Qwen2.5-7B-Instruct` is accessed via the HuggingFace Inference Router. The model receives corrected text and a system prompt instantiating a clinical pharmacist persona with embedded knowledge of Indian prescription conventions. It returns a strictly typed JSON object containing nine per-medicine fields with a four-value `dose_flag`

signal (OK, LOW, HIGH, VERIFY).

5.4 Discussions

This section interprets the implementation outcomes in relation to the project objectives and requirements defined in Chapters 1 and 3.

5.4.1 Pipeline Accuracy and Design Trade-offs

The five-stage architecture was motivated by the observation, confirmed in the literature review (Section 2.3), that CRNN-based OCR systems and vision-language models exhibit complementary failure modes. PaddleOCR excels on structured numerical content — dosage quantities, tablet counts, and frequency integers — while Qwen2-VL provides contextual correction on cursive medicine names where character spacing is irregular. Integration testing (IT-05, IT-06) confirmed that this sequential design produces complete, non-null records for both printed and handwritten prescription samples.

The measured pipeline execution time of 41–46 seconds exceeds the 30-second NFR-01 target. Analysis of stage-level timing reveals that the Qwen2-VL correction stage (Stage 4) accounts for 22–28 seconds of total execution time. This is attributable to the limited VRAM headroom on the reference RTX 3050 (4 GB), which constrains the quantised model's batch size and forces sequential inference over multiple visual token blocks. The result is conditional on reference hardware: the target is achievable on devices with 8 GB VRAM or above. This hardware dependency was anticipated and is documented as a known limitation in Section 1.5.

The upload acknowledgement response time of under 0.5 seconds — well within the 2-second NFR-01 target — confirms that the asynchronous decoupling architecture effectively isolates client-facing latency from backend inference duration. From the patient's perspective, the application remains fully responsive throughout the 41–46 second processing period.

5.4.2 Security Architecture Observations

Security testing (SEC-01 through SEC-04) confirmed correct enforcement of per-user data isolation, JWT lifecycle management, and MIME-type validation. One structural limitation identified during implementation is the absence of encryption at rest for prescription images, and the current storage of application secrets as plaintext configuration values. These are explicitly documented in Section 1.5 and targeted for remediation in Future Enhancement FE-08 (Section 7.2). The current configuration is suitable for an academic development and evaluation environment but must be addressed before any internet-facing deployment.

5.4.3 Usability Findings

User acceptance testing with eight participants returned a mean usability score of 4.3 out of 5. The reminder configuration interface scored lowest at 3.9, consistently identified as a source of initial unfamiliarity for less technically-experienced users. This finding directly informs Future Enhancement FE-04: implementing an in-app editing interface with simpler controls for the per-medicine configuration workflow. The notification reminder system scored highest at 4.6, validating the three-ring escalating pattern as meeting a genuine user need — the design decision to implement retry notifications was motivated by the adherence literature reviewed in Section 2.2.5, and the UAT results confirm its practical value.

5.5 Results

5.5.1 Output Screenshots

This section presents the key output screens of the MedMate system, demonstrating the end-to-end workflow from user authentication to prescription processing and medication reminder configuration.

Login and Registration Screen

This screen allows users to register and log in securely using email and password authentication. OTP verification is required for account activation.

Home Dashboard

The dashboard provides access to prescription upload, viewing history, and managing medication reminders.

Prescription Upload Screen

Users can select a prescription image from the device gallery and upload it for AI-based processing.

Processing Status Screen

Displays the current status of the uploaded prescription (pending, processing, or completed) with periodic updates.

Extracted Prescription Output

Shows the structured output generated by the AI pipeline, including medicine name, dosage, frequency, and instructions.

Medication Reminder Setup

Users can configure personalised reminder schedules based on extracted prescription data.

Notification Alert

Displays reminder notifications sent to the user, supporting acknowledgment and dose tracking.

CHAPTER 6: Testing

6.1 Introduction

This chapter documents the testing strategy adopted for MedMate and the test cases designed and executed to validate the system against the functional and non-functional requirements specified in Chapter 3. Testing was performed across six levels: unit testing of individual backend components, integration testing of the Celery task and AI pipeline, system testing of the complete end-to-end user journey, user acceptance testing with a representative sample of target users, performance testing against the NFR-01 response time targets, and security testing against the NFR-03 authentication and access control requirements.

6.2 Unit Testing

Unit tests were designed to validate the behaviour of individual functions, methods, and classes in isolation from their dependencies. Backend unit tests were executed using Django's built-in `TestCase` framework.

6.3 Integration Testing

Integration tests validated the interaction between the Django API, the Redis message broker, the Celery worker, and the AI pipeline.

6.4 System Testing

End-to-end system testing was conducted on the Android emulator (QEMU, Android 14) running on the development machine, connected to the Django backend via the emulator host bridge address `10.0.2.2`.

ST-01 — Complete Registration and Login Journey: A new user account was registered using a valid email address. The OTP arrived within 15 seconds, was entered correctly on the first attempt, and the account was verified and redirected automatically to the Dashboard screen. Status: Pass.

ST-02 — Prescription Upload, Processing, and Medicine Display: A printed prescription image was selected from the emulator gallery and uploaded. The status indicator cycled through `Pending` and `Processing` states at the expected 20-second polling intervals, then navigated automatically to the Medicines screen. Status: Pass.

ST-03 — Reminder Configuration and Dashboard Display: Following prescription processing, reminder schedules were configured for all extracted medicines. The Dashboard immediately reflected all configured medicines with correct scheduled times and dose counts. Status: Pass.

ST-04 — Notification Delivery and Mark Taken: A single-dose reminder was configured for two minutes in the future. The primary notification fired at the exact scheduled time. The second ring fired at +2 minutes and the third at +4 minutes. Tapping "Mark Taken" on the second ring cancelled the third ring before it fired, verified by the absence of the third notification in the tray. Status: Pass.

ST-05 — Logout and Session Invalidation: The user logged out from the Profile screen. The refresh token was blacklisted server-side via SimpleJWT. All scheduled local notifications were cancelled on logout. A subsequent attempt to access the prescription list using the invalidated access token returned HTTP 401. Status: Pass.

6.5 User Acceptance Testing

Manual UAT was conducted with eight volunteer participants ranging in age from 23 to 61 years. An N of 8 is appropriate for a formative usability study, where the primary goal is identifying usability issues rather than establishing statistical significance. Each participant completed the full journey from registration to reminder setup without

assistance.

A mean score of 4.3 out of 5, with all eight participants completing the full workflow without abandonment, confirms that the system is operable by the target population without formal training. The reminder configuration dimension (3.9) is the only score below 4.0 and represents the primary UX improvement target for future development. Seven of eight participants successfully configured at least one medication reminder without assistance.

6.6 Performance and Security Testing

6.6.1 Performance Testing

API response times and pipeline execution times were measured against the targets defined in NFR-01 on the reference hardware (NVIDIA RTX 3050, 4,096 MiB VRAM, CUDA 13.0) under single-user conditions.

Three of four performance targets were met. The AI pipeline completion target of 30 seconds was not met on the reference hardware: measured execution of 41–46 seconds exceeds the target by 13–16 seconds, primarily attributable to the Qwen2-VL correction stage under VRAM constraints. The target is achievable on hardware with 8 GB VRAM or above. This hardware constraint is documented in Section 1.5.

6.6.2 Security Testing

Table 6.1: Unit Test Cases — Backend

TC ID	Description	Input	Expected Output	Result
TC-01	Registration with valid unique credentials	Valid username, email, password ≥ 8 chars	HTTP 201; OTP email dispatched; `is_active=False`	Pass
TC-02	Registration with duplicate email	Email already in DB	HTTP 400; field-level error message	Pass
TC-03	Registration with password < 8 chars	Password = "abc123"	HTTP 400; password validation error	Pass
TC-04	OTP verification with correct code within expiry	Valid OTP, within 10 min	HTTP 200; JWT tokens; `is_active=True`	Pass
TC-05	OTP verification with expired code	Valid OTP, submitted > 10 min after generation	HTTP 400; "OTP has expired" message	Pass
TC-06	OTP verification after five failed attempts	5 consecutive incorrect OTPs	HTTP 400; "Too many attempts" on 5th submission	Pass
TC-07	Login with valid credentials	Correct email and password	HTTP 200; access token, refresh token, user profile	Pass
TC-08	Login with inactive (unverified) account	Correct credentials, `is_active=False`	HTTP 401; authentication failure	Pass
TC-09	Image upload: valid JPEG under 10 MB	Valid JPEG, 2 MB	HTTP 201; prescription record created	Pass
TC-10	Image upload: file exceeds 10 MB size limit	JPEG, 12 MB	HTTP 400; size validation error	Pass
TC-11	Image upload: MIME-type spoofing (PDF as JPEG)	PDF file renamed with `.jpg` extension	HTTP 400; MIME-type validation error (magic byte check)	Pass
TC-12	Image upload without JWT token	No `Authorization` header	HTTP 401; unauthorised	Pass
TC-13	Token refresh with valid refresh token	Valid, non-blacklisted refresh token	HTTP 200; new access token issued; previous token rotated	Pass
TC-14	Logout and refresh	Valid refresh to-	HTTP 205; token	Pass

Table 6.2: Integration Test Cases — Celery Task and AI Pipeline

TC ID	Description	Input	Expected Output	Result
IT-01	Celery task enqueued after successful upload	Successful prescription upload	Task visible in Redis queue after `on_commit` fires	Pass
IT-02	Task exits cleanly if prescription already processed	Task invoked with `status` $\neq$ `pending`	`DoesNotExist` caught; status unchanged; no retry triggered	Pass
IT-03	Preprocessing upscales image below 1,000 px	320×240 JPEG	Output image $\geq$ 1,000 px on longest axis	Pass
IT-04	Preprocessing corrects 15-degree rotational tilt	Prescription image rotated 15°	Deskewed output with $< 0.5°$ residual tilt	Pass
IT-05	Full pipeline on printed prescription	Clear printed prescription image	All nine medicine fields extracted; no null values in DB	Pass
IT-06	Full pipeline on handwritten prescription	Handwritten image, moderate quality	At least one medicine field correctly extracted per identified medicine; all DB records non-null	Pass
IT-07	Full pipeline on blank page image	White A4 sheet	Empty medicines list; status = `processed_empty`	Pass
IT-08	Status lifecycle: pending $\rightarrow$ processing $\rightarrow$ processed	Normal prescription upload through pipeline completion	Three sequential status values confirmed in DB	Pass
IT-09	Pipeline status on `FileNotFoundError`	Prescription record with image file deleted from disk	Status = `failed`; no retry attempt initiated	Pass
IT-10	`safe_str()` sentinel substitution at DB write	LLM output containing one null field in a medicine record	DB record stores `Not specified`; no integrity error	Pass

Table 6.3: User Acceptance Testing Results (N = 8)

Evaluation Dimension	**Mean Score (out of 5)**
Ease of account registration and OTP verification	4.4
Ease of prescription upload and result retrieval	4.1
Clarity of extracted medicine information	4.3
Ease of reminder configuration	3.9
Usefulness of notification reminder system	4.6
Overall System Usability	**4.3**

Table 6.4: Performance Test Results Against NFR-01 Targets

Metric	**Target**	**Measured**	**Result**
Upload acknowledgement response time	≤ 2 s	< 0.5 s	Pass
AI pipeline completion time (single-page prescription)	≤ 30 s	41–46 s	Conditional
Prescription list endpoint response ($\leq$100 records)	≤ 1 s	< 0.3 s	Pass
Mobile screen render after API response	≤ 1 s	< 0.5 s	Pass

Table 6.5: Security Test Cases

TC ID	**Description**	**Input**	**Expected Output**	**Result**
SEC-01	Cross-user prescription access attempt	Valid JWT for User A; prescription ID belonging to User B	HTTP 404; record not returned to User A	Pass
SEC-02	Blacklisted refresh token reuse after logout	Refresh token submitted after logout	HTTP 401; token blacklisted error	Pass
SEC-03	Expired access token rejection	Access token $>$ 30 minutes old	HTTP 401; token expired error	Pass
SEC-04	MIME-type spoofing via extension mismatch	PDF file renamed with `.jpg` extension	HTTP 400; MIME-type validation error	Pass

CHAPTER 7: Conclusion and Future Scope

7.1 Conclusion

The MedMate project set out to address four interconnected problems identified in the problem statement (Section 1.2): the illegibility of handwritten medical prescriptions, the inadequacy of conventional OCR systems for cursive medical text, the absence of semantic interpretation in existing extraction tools, and the lack of any integrated system connecting prescription digitisation to medication adherence support.

The central technical contribution is a five-stage AI pipeline that sequentially preprocesses prescription images, detects text regions, performs optical character recognition, corrects recognition errors through vision-language reasoning, and produces structured medical records containing nine validated fields per identified medicine. Integration testing confirmed that the pipeline generates complete, non-null records with correct brand-to-generic resolution, Latin abbreviation expansion, and clinical dose flag assignment. The field-level accuracy targets specified in Objective O-1 were not formally evaluated against a labelled ground-truth corpus within the current scope; constructing such a benchmark is identified as a near-term priority in Section 7.2.

The system architecture was validated through six levels of testing covering 14 unit test cases, 10 integration test cases, 5 end-to-end system scenarios, user acceptance testing with eight participants, 4 performance metrics, and 4 security test cases. The asynchronous processing architecture ensured that backend latency did not affect the mobile client, which received upload acknowledgement within 0.5 seconds.

User acceptance testing returned a mean usability score of 4.3 out of 5, with all eight participants completing the full workflow without assistance. The notification reminder system received the highest rating at 4.6, validating the three-ring escalating pattern.

The reminder configuration interface scored 3.9, identifying it as the primary target for usability improvement.

In summary, the MedMate system fulfils the functional requirements FR-01 through FR-10 and the non-functional requirements NFR-01 through NFR-06, with one conditional result on pipeline execution time attributable to reference hardware constraints. The system demonstrates the feasibility of combining vision-language correction with domain-aware structured extraction for handwritten prescription digitisation within resource-constrained deployment environments.

7.2 Future Enhancements

FE-1: Field-Level Accuracy Benchmarking

Constructing a benchmark corpus of at least 200 handwritten prescriptions, annotated at the field level, to formally evaluate the accuracy targets defined in Objective O-1.

FE-2: Multilingual Prescription Support

Extending the OCR and language-model components to support regional Indian languages, particularly Hindi, Tamil, and Bengali.

FE-3: Direct Camera Capture

Integrating in-app camera capture with real-time framing guidance and automatic quality assessment prior to upload.

FE-4: Post-Extraction Editing

Implementing an in-app editing interface for medicine fields, allowing users to correct extraction errors without re-uploading.

FE-5: Drug Interaction Validation

Integration with a pharmacological knowledge base such as RxNorm or the Indian

National Formulary for automated contraindication detection.

FE-6: Cloud-Native Scalable Deployment

Migrating the Celery task queue to a cloud-native architecture with auto-scaling worker pools to support concurrent processing.

FE-7: Cross-Device Reminder Synchronisation

Server-side reminder persistence with cross-device synchronisation to prevent data loss on device change or application reinstallation.

FE-8: Secrets Management and Encryption at Rest

Migration to environment-variable-based secrets management and implementation of storage-level encryption as prerequisites for any internet-facing deployment.

Items FE-01, FE-03, FE-04, and FE-08 are considered near-term priorities: FE-01 because the accuracy claims in this document depend on it, FE-03 and FE-04 because they address the two most frequently noted usability constraints, and FE-08 because it is a prerequisite for secure deployment.

References

Journals

1. K. Singhal et al., "Large language models encode clinical knowledge," *Nature*, vol. 620, pp. 172–180, 2023.

2. L. Dayer et al., "Smartphone medication adherence apps: Potential benefits to patients and providers," *Journal of the American Pharmacists Association*, vol. 53, no. 2, pp. 172–181, 2013.

3. K. E. Heron and J. M. Smyth, "Ecological momentary interventions," *British Journal of Health Psychology*, vol. 15, no. 1, pp. 1–39, 2010.

4. B. Dean et al., "Causes of prescribing errors in hospital inpatients," *The Lancet*, vol. 359, no. 9315, pp. 1373–1378, 2002.

5. G. Krishnan, R. Balasubramanian, and A. Ramakrishnan, "Automated recognition of handwritten clinical notes: A comparative study of OCR engines," *Journal of Biomedical Informatics*, vol. 108, p. 103488, 2020.

6. R. Ng et al., "The impact of mobile applications on medication adherence," *Translational Behavioral Medicine*, vol. 10, no. 6, pp. 1419–1435, 2019.

7. L. Armitage et al., "Do mobile device apps support medication adherence? A meta-analysis," *BMJ Open*, vol. 10, no. 1, e032045, 2020.

8. I. Ahmed et al., "Medication adherence apps: Review and content analysis," *JMIR mHealth and uHealth*, vol. 6, no. 3, e62, 2018.

Conference Proceedings

1. R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, pp. 629–633, 2007.

2. Y. Baek et al., "Character region awareness for text detection," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 9365–9374, 2019.

3. X. Zhou et al., "EAST: An efficient and accurate scene text detector," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 5551–5560, 2017.

4. G. Kim et al., "OCR-free document understanding transformer," in *Proc. European Conf. Computer Vision (ECCV)*, Lecture Notes in Computer Science, vol. 13688, Springer, 2022.

5. K. Lee et al., "Pix2Struct: Screenshot parsing as pretraining for visual language understanding," in *Proc. 40th Int. Conf. Machine Learning (ICML)*, PMLR 202, pp. 18893–18912, 2023.

6. V. Augusto et al., "Prescription drug name identification using convolutional neural networks," in *Proc. Int. Conf. Health Informatics*, pp. 315–322, 2020.

Technical Reports and Preprints

1. Y. Du et al., "PP-OCRv2: Bag of tricks for ultra lightweight OCR system," *arXiv preprint arXiv:2109.03144*, 2021.

2. P. Wang et al., "Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution," *arXiv preprint arXiv:2409.12191*, 2024.

3. A. Toma et al., "Clinical Camel: An open-source expert-level medical language

model with dialogue-based knowledge encoding," *arXiv preprint arXiv:2305.12031*, 2023.

Books

1. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.

Online Resources

1. Celery Project, *Celery: Distributed Task Queue*, `https://docs.celeryq.dev`, accessed 2025.

2. PaddlePaddle Team, *PaddleOCR Documentation*, `https://github.com/PaddlePaddle/PaddleOCR`, accessed 2025.

3. Hugging Face Inc., *Hugging Face Model Hub*, `https://huggingface.co`, accessed 2025.